Lab 03:
HW 3 (Single Cycle CPU)

Hunjun Lee
hunjunlee@hanyang.ac.kr

TODOs

- You will be implementing a single cycle CPU that you learned in the previous class
- Use the ALU and register file you made in the previous assignments
- The target architecture: MIPS
  - https://opencores.org/projects/plasma/opcodes
  - Do not need to implement every single instruction (implement what's provided in the following slides)
  - I'll provide the constant variables in a GLOBAL.v files

Target MIPS Assembly - 1

Assembly	Action	Opcode bitfields					
ADDU rd, rs, rt	rd = rs + rt	000000	rs	rt	rd	00000	100001
ADDIU rt, rs, imm	rt = rs + sign(imm)	001001	rs	rt	imm		
AND rd, rs, rt	rd = rs & rt	000000	rs	rt	rd	00000	100100
ANDI rt, rs, imm	rt = rs & zero(imm)	001100	rs	rt	imm		
LUI rt,imm	rt = imm << 16	001111	rs	rt	imm		
NOR rd,rs,rt	rd = ~(rs | rt)	000000	rs	rt	rd	00000	100111
OR rd,rs,rt	rd = rs | rt	000000	rs	rt	rd	00000	100101
ORI rt,rs,imm	rt = rs | zero(imm)	001101	rs	rt	imm		
SLT rd,rs,rt	rd = rs < rt	000000	rs	rt	rd	00000	101010
SLTI rt,rs,imm	rt = rs < sign(imm)	001010	rs	rt	imm		
SLTIU rt,rs,imm	rt = rs < sign(imm)	001011	rs	rt	imm		
SLTU rd,rs,rt	rd = rs < rt	000000	rs	rt	rd	00000	101011
SUBU rd,rs,rt	rd = rs - rt	000000	rs	rt	rd	00000	100011
XOR rd,rs,rt	rd = rs ^ rt	000000	rs	rt	rd	00000	101010
XORI rt,rs,imm	rt = rs ^ zero(imm)	001010	rs	rt	imm		

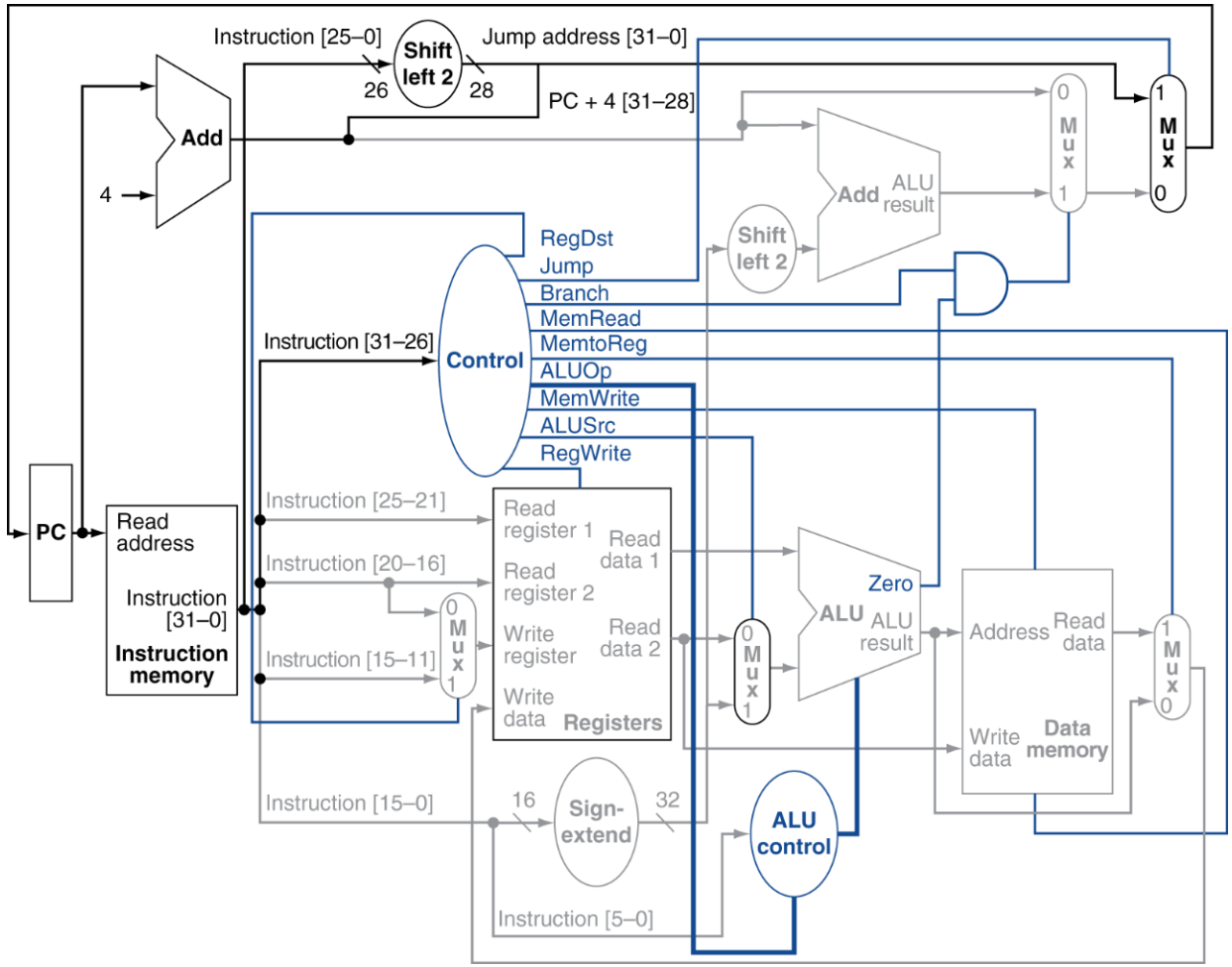
Target MIPS Assembly - 2

Assembly	Action	Opcode bitfields					
SLL rd, rt, sa	rd = rs << sa	000000	rs	rt	rd	sa	000000
SRA rd, rt, sa	rd = rt >> sa (arithmetic)	000000	rs	rt	rd	sa	000011
SRL rd, rt, sa	rd = rt >> sa (logical)	000000	rs	rt	rd	sa	000010
BEQ rs, rt, offset	if(rs == rt) pc += sign(offset) << 2	000100	rs	rt	offset		
BNE rs, rt, offset	if(rs != rt) pc += sign(offset) << 2	000101	rs	rt	offset		
J target	pc=pc_upper|(target << 2)	000010	target				
JAL target	r31 = pc; pc=pc_upper|(target << 2)	000011	target				
JR rs	pc = rs	000000	rs	000000000000000			001000
LW rt, offset(rs)	rt = MEM[sign(offset) + rs]	100011	rs	rt	offset		
SW rt, offset(rs)	MEM[sign(offset) + rs] = rt	101011	rs	rt	offset		

Implement a CPU that supports these ISAs

Target CPU Microarchitecture

- Implement what you learned in the class, but w/ **additional support for jal and jr**



Program Finish

- The program terminates when the PC reaches the end and the instruction becomes 32’b0
  - @CPU_TB.v

```
62
63      assign halt             = (inst == 32'b0);
64
```

Assignment

- **Files:**
  - **GLOBAL.v (provided)**
    - Include constant values (opcode, funct …)
  - **ALU.v (HW1)**
    - You can copy & paste the code from HW1
  - **RF.v (HW2)**
    - You can copy & paste the code from HW2
  - **MEM.v (provided)**
    - I did it for you
  - **CTRL.v (TODO)**
    - A controller to determine control signals
  - **CPU.v (TODO)**
    - The top module to combine these modules
  - **CPU_tb.v (provided)**
    - A testbench to test the CPU module

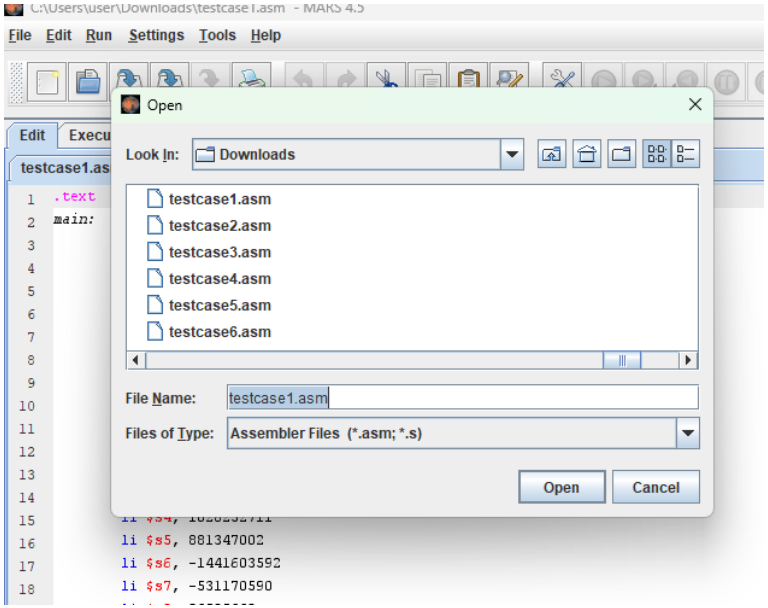
Memory module implementation

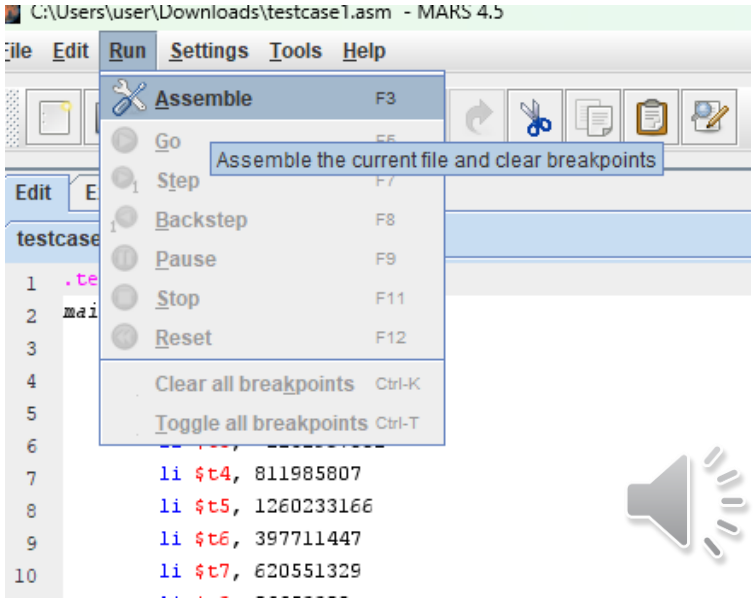
- I implemented a memory module that can simultaneously access instruction and data
- You will need to modify this code for multicycle CPU, but for now … use the provided file

```
3  module MEM(
4      input               clk,
5      input               rst,
6
7      input [31:0]        inst_addr,
8      output reg [31:0]   inst,
9
10     input [31:0]        mem_addr,
11     input               MemWrite,
12     input               MemRead,
13     input [31:0]        mem_write_data,
14     output reg [31:0]   mem_read_data
15     );
16
17     reg [31:0] memory [0:8191];
18
19     initial begin
20         $readmemh("initial_mem.mem", memory);
21     end
22
23     // Read instruction + memory data
24     always @(*) begin
25         inst = memory[(inst_addr >> 2)];
26         if (MemRead)
27             mem_read_data = memory[(mem_addr >> 2)];
28         else
29             mem_read_data = 32'hz;
30     end
31
32     always @(posedge clk) begin
33         if (!rst)
34             if (MemWrite)
35                 memory[(mem_addr >> 2)] <= mem_write_data;
36     end
37
38 endmodule
```

Debugging Verilog

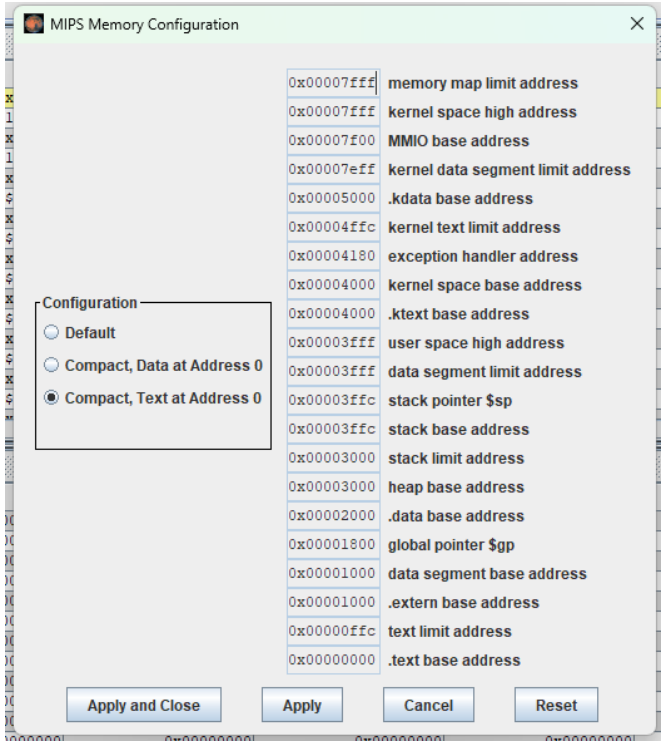
- **I provided a set of testcases to evaluate your Verilog project**
  - You will see seven different testcases
    - **asm:** MIPS assembly that runs on Mars simulator





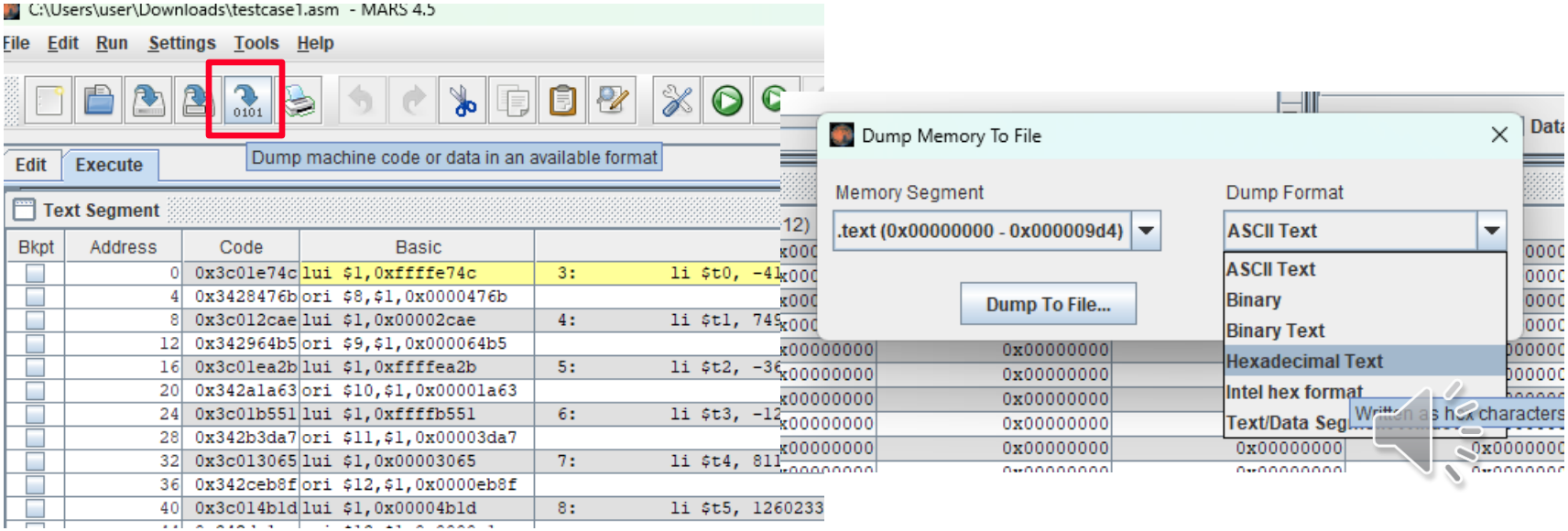
Debugging Verilog

- **I provided a set of testcases to evaluate your Verilog project**
  - You will see seven different testcases
    - **hex:** The instruction memory generated from Mars



Debugging Verilog

- **I provided a set of testcases to evaluate your Verilog project**
  - You will see seven different testcases
    - **hex:** The instruction memory generated from Mars



Debugging Verilog

- You can generate you own initial_reg and initial_mem files by running workload_gen.py
  - `python workload_gen.py testcase1.hex`
    - ➔ `This will generate initial_reg.mem & initial_mem.mem`

Debugging Verilog

- **I provided a set of testcases to evaluate your Verilog project**
  - You will see seven different testcases
    - **initial_reg.mem & initial_mem.mem:** The set of memory files that are loaded to the verilog

```
 3 module RF (
 4         input clk,
 5         input rst,
 6         // Read-related ports
 7         input [4:0] rd_addr1,
 8         input [4:0] rd_addr2,
 9         output reg [31:0] rd_data1,
10         output reg [31:0] rd_data2,
11         // Write-related ports
12         input RegWrite,
13         input [4:0] wr_addr,
14         input [31:0] wr_data
15     );
16
17     reg [31:0] register_file [0:31];
18
19     initial begin
20         $readmemh("initial_reg.mem", register_file);
21     end
22
```

RF.V

```
 1 `timescale 1ns / 1ps
 2
 3 module MEM(
 4     input               clk,
 5     input               rst,
 6
 7     input [31:0]        inst_addr,
 8     output reg [31:0]   inst,
 9
10     input [31:0]        mem_addr,
11     input               MemWrite,
12     input               MemRead,
13     input [31:0]        mem_write_data,
14     output reg [31:0]   mem_read_data
15     );
16
17     reg [31:0] memory [0:8191];
18
19     initial begin
20         $readmemh("initial_mem.mem", memory);
21     end
22
```

MEM.V

Debugging Verilog

- **I provided a set of testcases to evaluate your Verilog project**
  - You will see seven different testcases
    - **reference_reg.mem & reference_mem.mem:** The set of reference memory files after executing the data

```
 4 module CPU_tb;
 5     integer i;
 6     integer FAILED;
 7
 8     reg clk;
 9     reg rst;
10
11     wire halt;
12
13     // Have the reference register & mem
14     reg [31:0] register_file [0:31];
15     reg [31:0] memory [0:8191];
16
17     CPU cpu (.clk(clk), .rst(rst), .halt(halt));
18
19     initial begin : REF_INIT
20         $readmemh("reference_mem.mem", memory);
21         $readmemh("reference_reg.mem", register_file);
22     end
23
24
25     initial begin : CLOCK_GENERATOR
26         clk = 1'b0;
27         forever #5 clk = ~clk;
28     end
```

Submission

- Submission (Zip all the files)
  - For a two-people team: lab3_student_id1_student_id2.zip
  - For a one-person team: lab3_student_id.zip
    - You must follow the format (-10% for wrong file format)
    - Example:
      - lab3_2020102030.zip
      - lab3_2020102030_2022103040.zip
  - The zip file should contain:
    - every Verilog files except for the testbench
    - lab3_report.pdf

Submission

- You need to write a 4+-page report
  - Explain the overall structure and how you implemented each program
  - Write down the difficulties if you had one
  - Draw the hardware modules if needed …
  - How did you implement JAL / JR instruction and modified the uarchitecture?

- Due: Fri. April 24th
  - 1 week delay: -20%
  - Further delay: 0 point